Docker Compose Overview

Container Orchestration

Container orchestration is the **automated management** of containerized applications across multiple hosts, ensuring efficient deployment, scaling, networking, and management.

- **Automated Deployment & Scheduling** – Containers placed on the right nodes
- **Scaling** – Dynamically adjusts container numbers
- **Load Balancing** – Distributes traffic
- **Self-Healing** – Restarts failed containers
- **Service Discovery & Networking** – Manages inter-container communication
- **Security & Access Control** – Role-based access and secrets

Popular Tools

- Docker Compose (single-node)
- Docker Swarm (multi-node)
- Kubernetes (K8s)
- Red Hat OpenShift (K8s-based)

`docker-compose.yml` File

Docker Compose manages **multi-container Docker apps**. Its YAML file uses indentation to define services, networks, and volumes.

Key fields

- `services` : Each service = container
- `[service-name]`
    - `image` : Image to use
    - `build` : Build image from Dockerfile
    - `ports` : Map host to container ports
    - `volumes` : Mount storage
    - `networks` : Define connectivity
    - `depends_on` : Service dependencies
    - `environment` : Pass environment variables
    - `healthcheck` : Verify container health

Example: echo-server-logs

`docker-compose-simple.yml` :

```
services:
  echo:
    build: .
    ports:
      - "5000:5000"
```

Java controller:

```
@Log
@RestController
public class EchoController {
    @PostMapping(value = "/echo")
    public Map<String, Object> echo(@RequestBody String message) {
        log.info(message);
        return Map.of("echoed_data", message);
    }

    @GetMapping(value = "/logs")
    public Map<String, Object> logs() throws IOException {
        log.info("requested_logs");
        Path path = FileSystems.getDefault().getPath("/tmp", "application.log");
        return Map.of("lines", Files.readAllLines(path));
    }
}
```

Test:

```
mvn clean package -Dmaven.test.skip=true
docker compose -f docker-compose-simple.yml up --build --detach
```

```
curl -X POST http://localhost:5000/echo -H "Content-Type: application/json" -d '{"message": "Hello, Echo Server!"}'
curl -X GET http://localhost:5000/logs
```

Replicas

```
services:
  echo:
    build: .
    ports:
      - "5000"
    deploy:
      mode: replicated
      replicas: 3
```

```
mvn clean package -Dmaven.test.skip=true
docker compose -f docker-compose-replicas.yml up --build --detach
```

```
docker ps
curl -X POST http://localhost:32768/echo -H "Content-Type: application/json" -d '{"message": "Hello, Echo Server!"}'
```

Here's a concise explanation you can add to your slides or notes to clarify **what Docker volumes are** and the difference between the three types you listed:

Volumes

In Docker, a **volume** is a **persistent storage area** that exists outside of the container's writable layer. Volumes are used to:

- Preserve data between container restarts.
- Share data between multiple containers.
- Keep container images lightweight by storing large or mutable data externally.

Types of Volumes

1. **Named Volume**

```
services:
  echo:
    build: .
    ports:
      - "5000:5000"
    volumes:
      - tmp:/tmp

volumes:
  tmp:
```

- Stored and managed by Docker under `/var/lib/docker/volumes/` .
- Named volumes are **persistent** and survive container removal.
- Good for databases or other data that must outlive containers.

---

2. **Anonymous Volume**

```
services:
  echo:
    build: .
    ports:
      - "5000:5000"
    volumes:
      - /tmp
```

- Docker creates a volume automatically with a random name.
- Also persistent, but **harder to reference or reuse**.
- Useful for temporary storage where you don't care about the volume name.

---

3. **Bind Mount**

```
services:
  echo:
    build: .
    ports:
      - "5000:5000"
    volumes:
      - ./data:/tmp
```

- Maps a **host directory** to a directory inside the container.
- Changes in the container or host are immediately visible on both sides.
- Ideal for development, sharing code, or mounting configuration files.

Networks

Docker networks manage container connectivity.

- **Bridge:** Default, isolated from host
- **Host:** Shares host network
- **Overlay:** Multi-host communication
- **Macvlan:** Container appears as a physical device

Example: isolated frontend/backend networks

```
services:
  frontend:
    image: nginx
    volumes:
      - ./frontend/nginx.conf:/etc/nginx/nginx.conf
    networks:
      - front_net
    ports:
      - "8080:8080"
    depends_on:
      echo:
        condition: service_healthy

  echo:
    build: echo-server-logs-db-java
    environment:
      - SPRING_PROFILES_ACTIVE=docker
    networks:
      - front_net
      - back_net
    depends_on:
      postgres:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "curl -f http://localhost:5000/logs"]
      interval: 10s
      timeout: 5s
      retries: 5

  postgres:
    image: postgres:17-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: jdbc_schema
    volumes:
      - pg-data:/var/lib/postgresql/data
    networks:
      - back_net
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d jdbc_schema"]
      interval: 10s
      timeout: 5s
      retries: 5

networks:
  front_net:
    driver: bridge
  back_net:
    driver: bridge

volumes:
  pg-data:
```

```
cd echo-server-logs-db-java
mvn clean package -Dmaven.test.skip=true
cd ..
docker compose -f docker-compose-isolation.yam up --build --detach
```

Heartbeats / Health Checks

HTTP (curl)

```
healthcheck:
  test: ["CMD-SHELL", "curl -f http://localhost:8080/health"]
  interval: 30s
  timeout: 10s
  retries: 3
```

Database (Postgres)

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U user -d mydb"]
  interval: 30s
  timeout: 10s
  retries: 5
```

Redis

```
healthcheck:
  test: ["CMD-SHELL", "redis-cli ping"]
  interval: 30s
  timeout: 10s
  retries: 3
```

Integrate with depends_on

```
depends_on:
  db:
    condition: service_healthy
```

Resource Limitation

```
@RestController
public class MeterController {
    @GetMapping(value = "/")
    public Map<String,Object> echo() {
        long maxMemory = Runtime.getRuntime().maxMemory();
        long totalMemory = Runtime.getRuntime().totalMemory();
        long freeMemory = Runtime.getRuntime().freeMemory();
        return Map.of(
                "Available processors (cores)", Runtime.getRuntime().availableProcessors(),
                "Free memory (MB)", freeMemory / 1_000_000,
                "Maximum memory (MB)", maxMemory==Long.MAX_VALUE?"no limit":maxMemory/1_000_000,
                "Total memory (MB)", totalMemory/1_000_000,
                "Allocated memory (MB)", (totalMemory-freeMemory)/1_000_000
        );
    }

    @GetMapping(value = "/allocate/{size}")
    public Map<String,Object> allocate(@PathVariable Integer size) {
        try {
            byte[] array = new byte[size * 1_000_000];
            return Map.of("array allocation", "OK");
        } catch (OutOfMemoryError e) {
            return Map.of("array allocation", "Out of memory");
        }
    }
}
```

Compose resource limits:

```
services:
  meter:
    build: .
    ports:
      - "8080:8080"
    deploy:
      resources:
        limits:
          memory: 512M
          cpus: '2'
```

Test:

```
curl http://localhost:8080 | jq
curl http://localhost:8080/allocate/20
curl http://localhost:8080/allocate/200
```